CHAPTER 7: How Software Works?

Question 1: Advantages of Compiled vs. Interpreted Languages

Advantages of Compiled Languages (e.g., C, C++, Rust, Go):

Raw Execution Performance: Machine code runs directly on the bare metal CPU without runtime translation overhead.

Compile-Time Type & Syntax Verification: Catches type errors, missing references, and structural mistakes before deployment.

Optimized Resource Footprint: Produces compact, standalone binaries requiring less runtime memory and no separate runtime environment.

Advantages of Interpreted Languages (e.g., Python, JavaScript, Ruby):

~~Rapidd~~ Rapid Development & Prototyping: ~~Ruun~~ Run code immediately upon saving without waiting for compilation cycles.

Cross-Platform Portability: The exact same script runs on Linux, macOS, or Windows as long as the host interpreter is installed.

Interactive REPL & Dynamic Features: Supports dynamic typing, runtime code evaluation (eval), and immediate interactive debugging.

Question 2: "Translation always happens. The only question is when, and who does it."

Hardware CPUs are incapable of understanding human-readable code; silicon logic gates strictly execute raw binary machine instructions (opcodes). Therefore, translation from high-level syntax into electrical instruction words is non-negotiable.

The Compiler: Translates the entire program ahead of time (before runtime). The developer runs a compiler that translates source code into a ~~standalonne~~ standalone binary file of machine instructions. The CPU later executes that pre-translated binary directly at full hardware speed.

The Interpreter: Translates the code ~~line-b-ytine~~ line-by-line during runtime. An active host program (the interpreter) reads the high-level statements, analyzes their meaning on the fly, and translates them into machine operations while the program is actively executing.